

Estructura de datos: matriz dispersa con listas enlazadas cruzadas

Una hoja de cálculo tiene potencialmente millones de celdas, pero solo una fracción pequeña contiene datos. Una matriz densa desperdiciaría memoria proporcional al tamaño total de la grilla. La estructura elegida es una **lista enlazada cruzada** (*cross-linked list*):

- Cada celda con contenido es un nodo con punteros **right**, **left**, **down** y **up**.
- Existe un **RowHeader** por cada fila activa y un **ColHeader** por cada columna activa, organizados en listas enlazadas que permiten crecimiento dinámico.
- Solo se crean nodos para celdas con contenido, por lo que la memoria es proporcional a las celdas ocupadas, no al tamaño de la grilla.

Complejidad de las operaciones

Sean R el número de filas activas, C el de columnas activas, k_r las celdas en una fila, k_c las celdas en una columna y N el total de celdas ocupadas.

Operación	Complejidad	Justificación
<code>set(row, col, val)</code>	$O(R + C)$	Buscar o crear RowHeader en $O(R)$, ColHeader en $O(C)$, insertar en la fila en $O(k_r)$.
<code>get(row, col)</code>	$O(R + k_r)$	Buscar RowHeader en $O(R)$, recorrer celdas de la fila en $O(k_r)$.
<code>clear_cell(row, col)</code>	$O(R + k_r)$	Igual que get , más actualizar punteros vecinos en $O(1)$.
<code>delete_row(row)</code>	$O(R + k_r \cdot C)$	Encontrar la fila en $O(R)$, desenlazar cada celda de su columna en $O(C)$.
<code>delete_col(col)</code>	$O(C + k_c \cdot R)$	Simétrico a delete_row .
<code>delete_range(...)</code>	$O(\Delta r \cdot \Delta c \cdot \max(R, C))$	Recorre el rango y borra cada celda con delete_cell_node . Agrupa los cambios en un único Action ; deshacer cuenta como una sola operación de undo.
<code>set_batch(cells)</code>	$O(k \cdot (R + C))$	k escrituras, cada una en $O(R + C)$. Agrupa los cambios en un único Action ; deshacer cuenta como una sola operación de undo.
<code>for_each</code>	$O(N)$	Recorre todos los nodos existentes.
<code>undo / redo</code>	$O(k)$	k = número de cambios en la acción a revertir.

En comparación, una matriz densa tendría **get/set** en $O(1)$ pero ocuparía $O(n \cdot m)$ de memoria, y **delete_row/delete_col** requerirían $O(n \cdot m)$ para desplazar elementos.

Evaluación de fórmulas

Las fórmulas se almacenan como strings y se evalúan al momento de mostrarlas (evaluación *lazy*). Esto evita mantener un grafo de dependencias explícito y recalculando celdas en cascada al modificar un valor.

La detección de ciclos usa un `std::set<CellCoord>` que representa el stack de llamadas activo. Al entrar a resolver una celda se inserta su coordenada, y al retornar se elimina (backtracking). Si al resolver una referencia la celda destino ya está en el conjunto, significa que hay un ciclo real y se devuelve `#CICLO`.